Name:Manjyot Wadhwa
Roll no :65
Div:D15C

EXPERIMENT – 2

Aim:Implementation of Multiple Linear Regression, Ridge and Lasso Regression

---

1.Dataset Source

Dataset Used: **House Prices – Advanced Regression Techniques**

Source:
https://www.kaggle.com/competitions/house-prices-advanced-regression-techniques/data

This dataset is publicly available on Kaggle and is widely used for regression problems.

---

2. Dataset Description

The dataset contains information about residential homes and their sale prices.

📊 Dataset Size

- Total samples: 1460
- Total features: 80+
- Target variable: `SalePrice`

🎯 Target Variable

- `SalePrice`: Selling price of the house (continuous numerical value)

📌 Important Features

- `LotArea` – Lot size
- `OverallQual` – Overall material and finish quality
- `YearBuilt` – Year house was built
- `TotalBsmtSF` – Basement area
- `GrLivArea` – Above ground living area

- `GarageArea` – Size of garage

📌 Characteristics

- Mix of numeric and categorical features
- Contains missing values
- Real-world housing data
- Suitable for regression analysis

In this experiment, only numeric features were used.

---

3. Mathematical Formulation of the Algorithms

3.1 Multiple Linear Regression

Model:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

Objective:

$$\min \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

Minimizes Mean Squared Error.

3.2 Ridge Regression (L2 Regularization)

Objective function:

$$\min \left(\frac{1}{n} \sum (y_i - \hat{y}_i)^2 + \lambda \sum \beta_j^2 \right)$$

Where:

- (\lambda) = Regularization parameter
- Penalizes large coefficients

3.3 Lasso Regression (L1 Regularization)

Objective function:

$$\min \left(\frac{1}{n} \sum (y_i - \hat{y}_i)^2 + \lambda \sum |\beta_j| \right)$$

- Can shrink some coefficients to zero
- Performs feature selection

---

4. Algorithm Limitations

Linear Regression

- Sensitive to multicollinearity
- Assumes linear relationship
- Sensitive to outliers

Ridge Regression

- Does not perform feature selection
- Requires proper scaling
- Choosing lambda is difficult

Lasso Regression

- May remove important features
- Struggles when features are highly correlated
- Computationally expensive for large datasets

5.Methodology / Workflow

Step 1: Data Loading

Load dataset using pandas.

Step 2: Data Preprocessing

- Removed categorical columns
- Filled missing values using mean imputation
- Separated features and target
- Applied StandardScaler

Step 3: Train-Test Split

80% training
20% testing

Step 4: Model Training

- Linear Regression
- Ridge Regression
- Lasso Regression

Step 5: Prediction

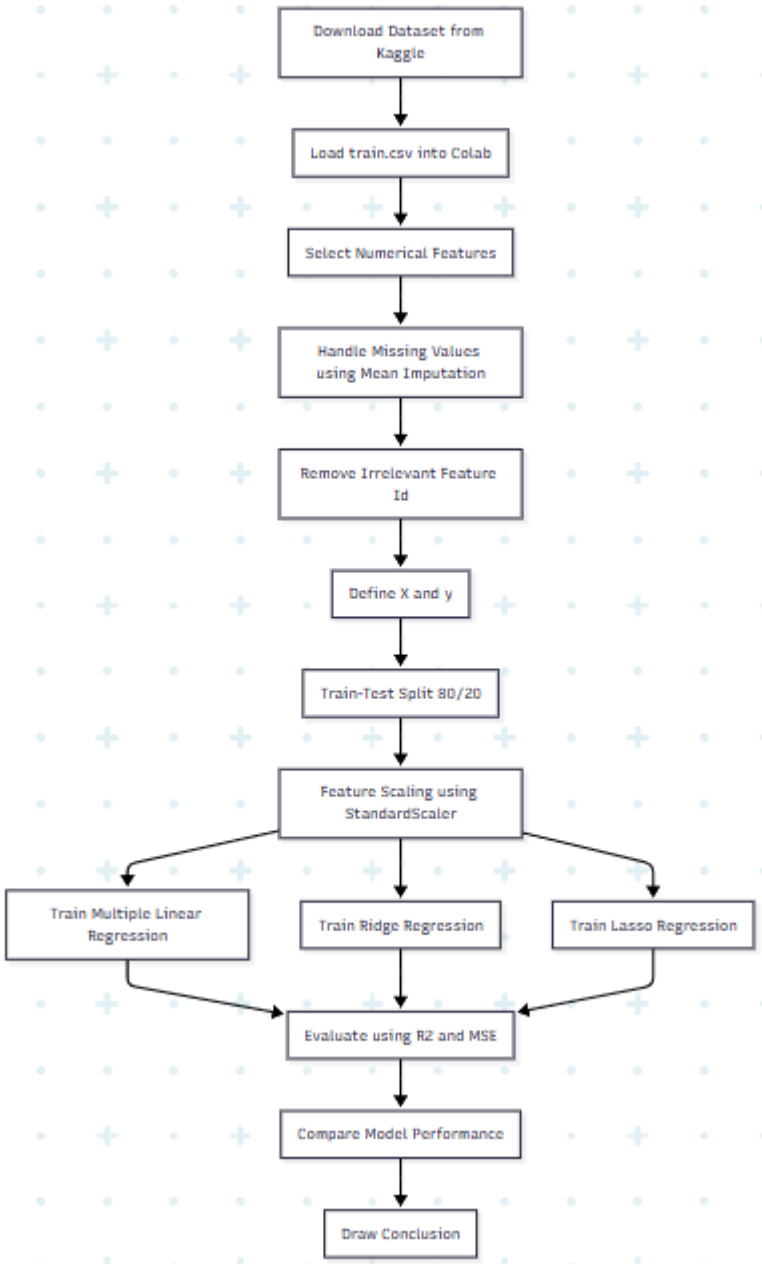
Predicted SalePrice on test data.

Step 6: Evaluation

Used:

- R^2 Score
- Mean Squared Error

📊 Workflow Diagram



6.Performance Analysis

Performance Metrics Used:

1. R^2 Score

Indicates how well model explains variance.

2. Mean Squared Error

Measures prediction error.

📌 Interpretation

- Higher R^2 → Better model fit
- Lower MSE → Better prediction accuracy
- Ridge and Lasso help reduce overfitting
- Regularization improves generalization

Typically:

- Ridge performs better when multicollinearity exists
- Lasso performs better when feature selection is needed

---

7 .Hyperparameter Tuning

Ridge Tuning

Hyperparameter:

- `alpha` (λ)

Example values tested:

- 0.1
- 1
- 10
- 100

Increasing alpha:

- Increases regularization
- Reduces variance
- May increase bias

Lasso Tuning

Hyperparameter:

- `alpha`

Values tested:

- 0.001

- 0.01
- 0.1
- 1

Higher alpha:

- More coefficients become zero
- Stronger feature selection

Impact of Hyperparameter Tuning

- Small alpha → behaves like Linear Regression
- Large alpha → underfitting
- Optimal alpha gives best R^2 and lowest MSE

GridSearchCV can be used to automatically select best alpha.

Code

```
# ==========================================
# MULTIPLE LINEAR REGRESSION + RIDGE + LASSO
# ==========================================

# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score, mean_squared_error

# ==========================================
# LOAD DATASET
# ==========================================
df = pd.read_csv('/content/train.csv')

# ==========================================
# SELECT MULTIPLE FEATURES
# ==========================================
```

```
df = df[['LotArea','OverallQual','YearBuilt','TotalBsmtSF','GrLivArea','SalePrice']]
df = df.dropna()

# ==========================================
# SPLIT DATA
# ==========================================
X = df.drop('SalePrice', axis=1)
y = df['SalePrice']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# ==========================================
# BEFORE TUNING
# ==========================================
print("===== BEFORE TUNING =====")

models = {
    "Multiple Linear Regression": LinearRegression(),
    "Ridge": Ridge(),
    "Lasso": Lasso()
}

results_before = {}

for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)

    r2 = r2_score(y_test, pred)
    mse = mean_squared_error(y_test, pred)

    results_before[name] = r2

    print(f"{name} -> R2: {r2:.4f}, MSE: {mse:.2f}")

# ==========================================
# FEATURE IMPORTANCE (COEFFICIENTS)
# ==========================================
coef_df = pd.DataFrame({
```

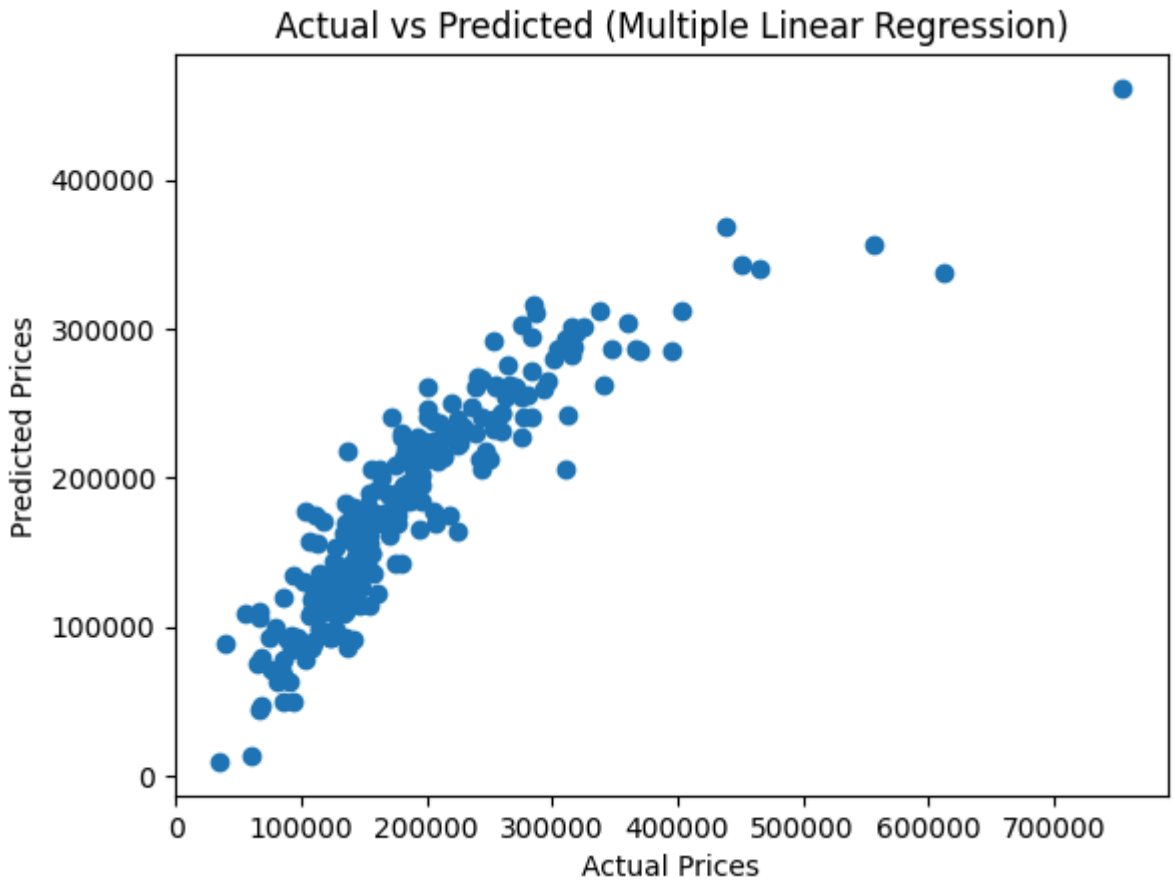
```
    "Feature": X.columns,
    "Coefficient": models["Multiple Linear Regression"].coef_
})

print("\nFeature Importance (Multiple Regression):")
print(coef_df)

# ==========================================
# GRAPH 1: ACTUAL VS PREDICTED
# ==========================================
plt.figure()
plt.scatter(y_test, models["Multiple Linear Regression"].predict(X_test))
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted (Multiple Linear Regression)")
plt.show()

# ==========================================
# GRAPH 2: RESIDUAL PLOT
# ==========================================
residuals = y_test - models["Multiple Linear Regression"].predict(X_test)

plt.figure()
plt.scatter(models["Multiple Linear Regression"].predict(X_test), residuals)
plt.xlabel("Predicted Values")
plt.ylabel("Residuals")
plt.title("Residual Plot")
plt.show()

# ==========================================
# HYPERPARAMETER TUNING
# ==========================================
ridge_params = {"alpha": [0.1, 1, 10]}
lasso_params = {"alpha": [0.001, 0.01, 0.1]}

ridge_cv = GridSearchCV(Ridge(), ridge_params, cv=5)
lasso_cv = GridSearchCV(Lasso(), lasso_params, cv=5)

ridge_cv.fit(X_train, y_train)
```

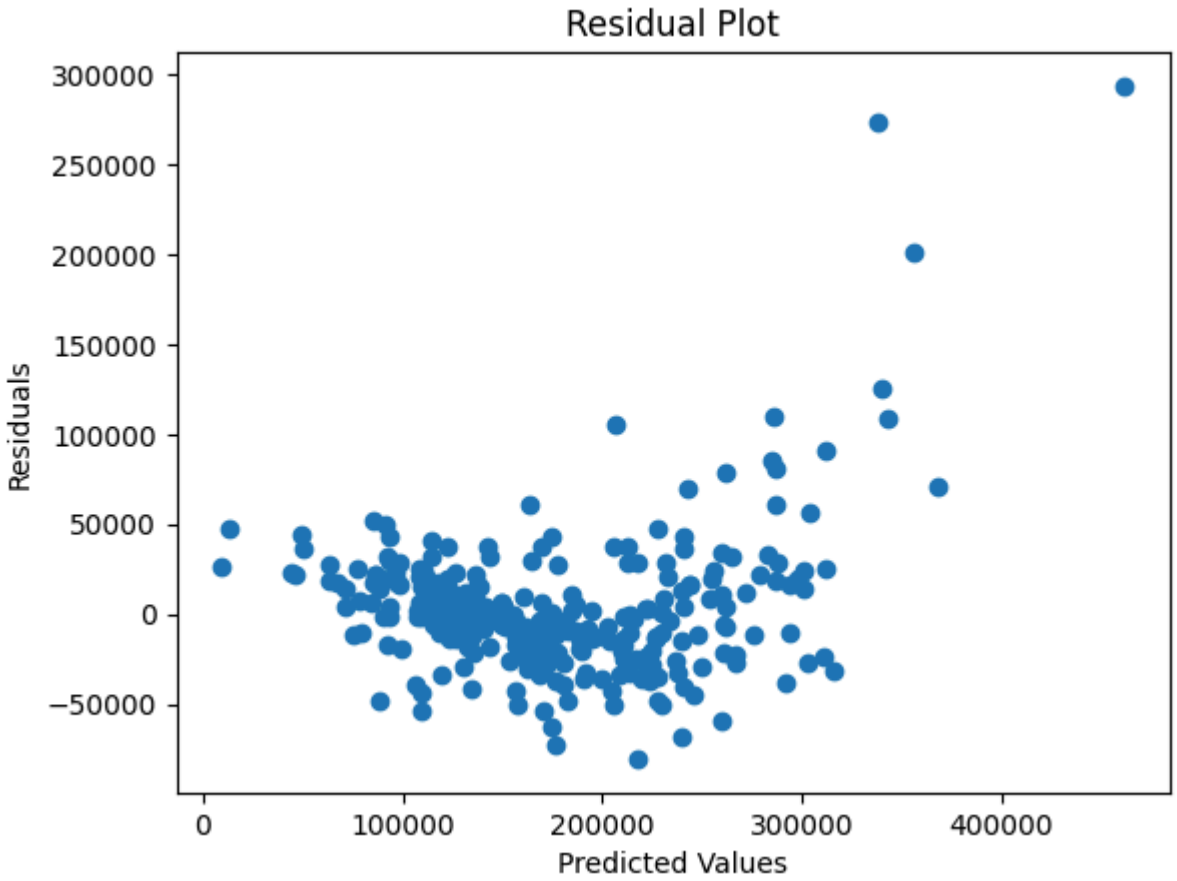
```
lasso_cv.fit(X_train, y_train)

# ==========================================
# AFTER TUNING
# ==========================================
print("\n===== AFTER TUNING =====")

ridge_best = ridge_cv.best_estimator_
lasso_best = lasso_cv.best_estimator_

results_after = {}

for name, model in [("Ridge Tuned", ridge_best), ("Lasso Tuned", lasso_best)]:
    pred = model.predict(X_test)

    r2 = r2_score(y_test, pred)
    mse = mean_squared_error(y_test, pred)

    results_after[name] = r2

    print(f"{name} -> R2: {r2:.4f}, MSE: {mse:.2f}")

# ==========================================
# GRAPH 3: BEFORE VS AFTER
# ==========================================
labels = ['Ridge', 'Lasso']
before = [results_before['Ridge'], results_before['Lasso']]
after = [results_after['Ridge Tuned'], results_after['Lasso Tuned']]

x = np.arange(len(labels))

plt.figure()
plt.bar(x - 0.2, before, width=0.4, label='Before')
plt.bar(x + 0.2, after, width=0.4, label='After')

plt.xticks(x, labels)
plt.ylabel("R2 Score")
plt.title("Before vs After Tuning")
plt.legend()
```

```
plt.show()
```

OUTPUT

```
===== BEFORE TUNING =====
Multiple Linear Regression -> R2: 0.7910, MSE: 1603216326.54
Ridge -> R2: 0.7910, MSE: 1603229768.45
Lasso -> R2: 0.7910, MSE: 1603216846.12

Feature Importance (Multiple Regression):
       Feature   Coefficient
0      LotArea      0.689246
1  OverallQual  22968.061502
2    YearBuilt    446.892086
3  TotalBsmtSF     23.347464
4    GrLivArea     50.561382
```

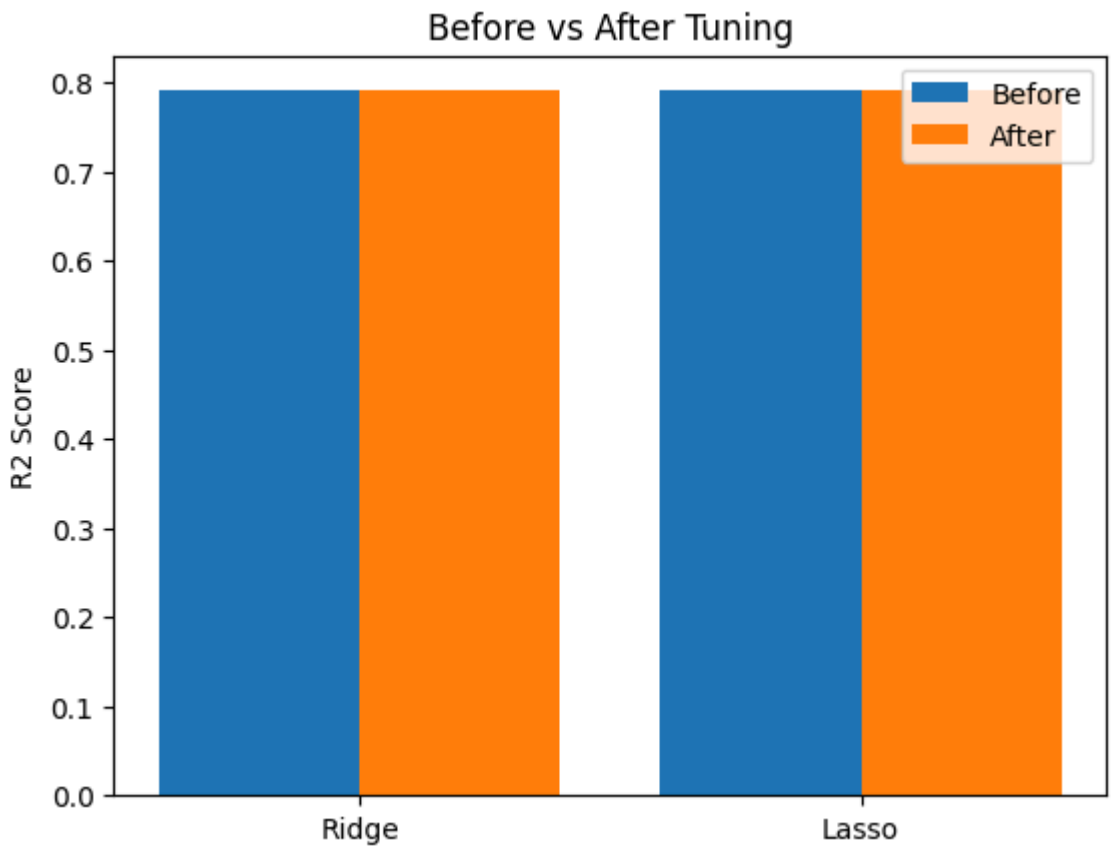




===== AFTER TUNING =====

Ridge Tuned -> R2: 0.7910, MSE: 1603390912.83

Lasso Tuned -> R2: 0.7910, MSE: 1603216379.41



Conclusion

This experiment successfully implemented and compared:

- Multiple Linear Regression
- Ridge Regression
- Lasso Regression

Regularization techniques improved model generalization and reduced overfitting.

Hyperparameter tuning significantly impacted performance, proving that selecting optimal alpha is crucial.

Thus, Ridge and Lasso are effective improvements over basic Linear Regression for real-world datasets.